

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ
УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
«КИЄВО-МОГИЛЯНСЬКА АКАДЕМІЯ»**

Факультет інформатики

Кафедра інженерії програмного забезпечення

ЗВІТ

ПРО ВИКОНАННЯ НАВЧАЛЬНОЇ ПРАКТИКИ

З КУРСУ «ПРАКТИКА НАВЧАЛЬНА. ПРОГРАМУВАННЯ JAVA»

На тему: «Піксельна вишивка. Редактор орнаменту»

Виконав:

Студент групи ІПЗ-1
спеціальності «Інженерія програмного забезпечення»
Різник Нікіта Юрійович

Перевірив:

Асистент / Професор кафедри

Київ – 2026

ЗМІСТ

1. Постановка задачі
2. Структура програми та архітектурні рішення
3. Опис класів та методів
4. Розподіл ролей та етапи розробки
5. Інструкція користувача
6. Семантичний словник традиційних українських орнаментів
7. Висновок
8. Лістинг програмного коду

СПИСОК ІЛЮСТРАЦІЙ

- Рис. 1.1. Головне вікно застосунку «Vyshyvanka» під час запуску та автоматичної генерації патерну імені.
- Рис. 1.2. Інструменти палітри та налаштування розмірності інтерактивної сітки.
- Рис. 1.3. Схема застосування дзеркальної симетрії по осях X, Y та XY.
- Рис. 1.4. Приклад експортованого файлу орнаменту у форматі PNG.

1. ПОСТАНОВКА ЗАДАЧІ

Метою навчальної практики є отримання практичних навичок роботи з подіями, графічним інтерфейсом користувача (GUI) та динамічною побудовою растрових зображень засобами мови Java.

В межах індивідуального завдання розроблено десктопний застосунок для інтерактивного створення, модифікації та автоматичної генерації схем української народної вишивки.

Програма реалізує наступні технічні вимоги:

1. **Інтерактивний редактор:** можливість вільного вибору кольорів із визначеної палітри та малювання за допомогою миші на дискретній сітці клітинок.
2. **Внутрішня структура даних:** збереження поточного стану полотна у вигляді оптимізованої матриці кольорів.
3. **Автоматична генерація за текстом:** при першому запуску програма зчитує ім'я користувача та автоматично створює закодовану автентичну схему вишивки.
4. **Алгоритми симетрії:** автоматична розбудова орнаментів із застосуванням горизонтального, вертикального та комбінованого віддзеркалення, а також циклічного дублювання базових фрагментів.
5. **Модуль збереження:** експорт готових результатів у растрові графічні файли формату PNG, а також збереження/завантаження проектів у текстовому форматі JSON для подальшого редагування.

2. СТРУКТУРА ПРОГРАМИ ТА АРХІТЕКТУРНІ РІШЕННЯ

Проект побудовано на базі архітектурного шаблону розділення відповідальності. Управління залежностями й збірка виконуються за допомогою Maven. Застосунок використовує версію Java 21.

Пакетна структура кодової бази проекту:

- `ui` — містить класи графічного інтерфейсу, класи обробки подій периферійних пристроїв та логіку текстової генерації патернів.
- `serialization` — відповідає за збереження даних, серіалізацію об'єктів та ручний парсинг файлів конфігурації.

Важливим архітектурним рішенням є відмова від сторонніх важких бібліотек (на кшталт Jackson або GSON) на користь написання власного легковагового детермінованого парсера JSON. Це гарантує повний контроль над валідацією схем даних та оптимізує швидкість роботи з файловою системою.

3. ОПИС КЛАСІВ ТА МЕТОДІВ

3.1. Пакет `ui`

Клас `ui.Main`

Є точкою входу в застосунок. Наслідує клас `javax.swing.JFrame`.

- `main(String[] args)` — ініціалізує потік обробки подій Swing (`SwingUtilities.invokeLater()`).
- `initializeUI()` — конструює основне вікно, створює панель інструментів (колірна палітра з 16 автентичних відтінків, елементи `JSpinner` для керування розмірами сітки клітинок).
- `setupMenu()` — створює головне меню верхнього рівня `JMenuBar` (File -> Open, Save, Export PNG).

Клас `ui.EmbroideryCanvas`

Основний інтерактивний UI-компонент, що наслідує `javax.swing.JPanel`. Відповідає за відображення сітки та обробку малювання.

- `paintComponent(Graphics g)` — перевизначений метод життєвого циклу Swing для рендерингу двовимірної матриці клітинок та допоміжних ліній сітки.
- `drawPixel(int mouseX, int mouseY)` — вираховує індекси матриці на основі екранних координат курсору та встановлює поточний активний колір із палітри, враховуючи налаштований режим симетрії.
- `resizeGrid(int newRows, int newCols)` — динамічно масштабує полотно, виконуючи безпечне копіювання існуючих значень кольорів у новий масив без втрати даних користувача.
- `toImage(boolean includeGrid)` — генерує об'єкт `BufferedImage` для збереження схеми у файл.

Клас `ui.TextEmbroideryGenerator`

Реалізує математичні алгоритми кодування алфавітно-цифрових символів у автентичні графічні образи.

- `generatePatternForText(String text)` — верхньорівневий метод, що ініціює створення орнаменту.
- `getMotifForCharacter(char ch)` — трансформує кожен окрему літеру в двійковий масив

розміром 5x5 через побітові маски.

- `createFlower(char centralChar)` — створює збільшений центральний графічний елемент («квітку») розміром 9x9 для першої літери слова шляхом квадрантного віддзеркалення її базового мотиву.

3.2. Пакет `serialization`

Рекорд `serialization.EmbroideryPattern`

Сучасна незмінна структура даних Java (record), яка слугує як Data Transfer Object (DTO) для збереження стану. Зберігає версію файлу схеми, кількість рядків, колонок та плоский або двовимірний ряд HEX-кодів кольорів (`String[][]`).

Клас `serialization.PatternSerializer`

Клас, що забезпечує низькорівневий запис текстових стримів та містить вбудований інтерпретатор об'єктів JSON.

- `save(EmbroideryPattern pattern, File file)` — серіалізує об'єкт в структурований текстовий файл конфігурації.
- Внутрішній клас `JsonParser` — посимвольно аналізує синтаксис файлу за допомогою методів `readObject()`, `readArray()`, `readString()`, відкидаючи зайві пробіли через логічний фільтр `skipWhitespace()`.

4. РОЗПОДІЛ РОЛЕЙ ТА ЕТАПИ РОЗРОБКИ

Даний проєкт є індивідуальною роботою, виконаною студентом Різником Нікітою одноосібно. Весь життєвий цикл розробки ПЗ (SDLC) був декомпонований на наступні послідовні спринти:

1. **Аналіз вимог та проектування архітектури:** Дослідження технічного завдання практики, вивчення принципів побудови піксельних редакторів, проектування структури класів та модульних пакетів.
2. **Розробка графічного рушія (UI Модуль):** Створення класу `EmbroideryCanvas`, імплементація подій миші для забезпечення плавного безперервного малювання затиснутою кнопкою курсору.
3. **Імплементація алгоритмів симетрії:** Математичне моделювання віддзеркалення координат на дискретному полі. Написання та тестування функцій дзеркального копіювання клітинок у реальному часі.
4. **Алгоритмізація текстового генератора:** Розробка унікальних шрифтових двійкових матриць 5x5 для української абетки, створення механізму розгортання «центральної квітки» та спірального пакування символьних токенів довкола центру.
5. **Розробка підсистеми збереження даних:** Написання власного JSON-парсера з нуля, імплементація експорту схем у графічний формат PNG за допомогою утиліт `javax.imageio.ImageIO`.
6. **Тестування та рефакторинг:** Покриття розробленого функціоналу набором JUnit тестів для перевірки цілісності даних при завантаженні/збереженні та коректності генерації геометричних фігур.

5. ІНСТРУКЦІЯ КОРИСТУВАЧА

1. **Запуск програми:** При першому запуску застосунку на екрані з'являється інтерактивне полотно, на якому автоматично згенеровано унікальний орнамент імені користувача.
2. **Вільне малювання:** Для малювання власного візерунка оберіть лівою кнопкою миші будь-який колір із палітри, розташованої на лівій бічній панелі програми. Затисніть ліву кнопку миші на полотні сітки клітинок та ведіть курсор для створення ліній. Для видалення помилково зафарбованої клітинки виберіть білий колір палітри (ластик).
3. **Налаштування розмірів полотна:** За допомогою інтерактивних лічильників (JSpinner) у верхній частині інтерфейсу користувач може встановити кількість рядків та стовпців сітки, а також відрегулювати масштаб клітинки у пікселях для комфортного малювання.
4. **Використання симетрії:** У випадяючому списку інструментів оберіть один із доступних режимів автоматичного віддзеркалення: «Horizontal», «Vertical» або «XY-Symmetry». Будь-яка поставлена точка буде автоматично дублюватися у відповідних координатах інших квадрантів полотна.
5. **Збереження та експорт:**
 - Для збереження робочої сесії з можливістю подальшого редагування перейдіть у меню **File -> Save Project**, та вкажіть назву файлу.
 - Для експорту готової схеми у вигляді зображення для друку виберіть пункт меню **File -> Export PNG**.

6. СЕМАНТИЧНИЙ СЛОВНИК ТРАДИЦІЙНИХ УКРАЇНСЬКИХ ОРНАМЕНТІВ

З метою інтеграції культурної спадщини в алгоритми генератора візерунків було досліджено традиційну сакральну геометрію української народної вишивки:

- **Квадрат:** Базовий геометричний елемент, що уособлює благополуччя, земний мир, достаток, гармонію та є символом засіяного, захищеного поля.
- **Ромб:** Символ союзу сонця і землі, знак плодючості та продовження роду. **Ромб із крапкою** всередині традиційно позначає засіяне зерном родюче поле.
- **Коло:** Знаменник сонця, божественної творчої енергії, безперервності буття, циклічності часу та вічності.
- **Прямий Хрест:** Потужний прадавній оберіг від злих духів, що закриває людину з чотирьох сторін світу.
- **Восьмипроменева Зірка (Алатир):** Символ поєднання жіночого (косого хреста) та чоловічого (прямого хреста) начал, що дає початок усьому живому. Вона слугує оберегом від зла і негативу, захищає від хвороб.
- **Рослинні Мотиви:**
 - *Калина* — відображає любов, багатство, жіночу красу, материнство та єдність роду.
 - *Гілки дуба* — є уособленням Перуна (бога грому), уособлюють мужність, незламну чоловічу силу та захист.
 - *Виноград* — традиційний символ щасливого сімейного благополуччя.

7. ВИСНОВОК

Під час виконання навчальної практики було спроектовано та успішно реалізовано десктопний застосунок «Vyshyvanka» мовою Java з використанням бібліотеки графічних компонентів Swing.

В ході роботи закріплено навички побудови графічних інтерфейсів користувача (GUI), обробки подій миші та динамічної роботи з двовимірними матрицями та масивами даних. Реалізовані складні алгоритми геометричної симетрії по різних осях координат дозволяють створювати комплексні схеми орнаментів на основі невеликих фрагментів. Програма містить модуль генерації автентичних патернів вишивки імені на базі розробленої системи побітового кодування символів української абетки. Написання власного парсера серіалізації в JSON дозволило поглибити знання з обробки текстових потоків даних, валідації регулярними виразами та управління винятковими ситуаціями в Java.

8. ЛІСТИНГ ПРОГРАМНОГО КОДУ

8.1. Основний клас ui.EmbroideryCanvas

```
Java
package ui;

import javax.swing.*;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.image.BufferedImage;

public class EmbroideryCanvas extends JPanel {
    private int rows;
    private int cols;
    private int cellSize;
    private Color[][] grid;
    private Color currentColor = Color.RED;
    private String symmetryMode = "None"; // "None", "Horizontal", "Vertical", "XY"

    public EmbroideryCanvas(int rows, int cols, int cellSize) {
        this.rows = rows;
        this.cols = cols;
```

```
this.cellSize = cellSize;  
this.grid = new Color[rows][cols];  
clearGrid();
```

```
MouseListener mouseHandler = new MouseAdapter() {  
    @Override  
    public void mousePressed(MouseEvent e) {  
        drawPixel(e.getX(), e.getY());  
    }  
}
```

```
    @Override  
    public void mouseDragged(MouseEvent e) {  
        drawPixel(e.getX(), e.getY());  
    }  
};  
addMouseListener(mouseHandler);  
addMouseMotionListener(mouseHandler);  
}
```

```
public void clearGrid() {  
    for (int r = 0; r < rows; r++) {  
        for (int c = 0; c < cols; c++) {  
            grid[r][c] = Color.WHITE;  
        }  
    }  
    repaint();  
}
```

```
private void drawPixel(int mouseX, int mouseY) {  
    int c = mouseX / cellSize;  
    int r = mouseY / cellSize;
```

```
    if (r >= 0 && r < rows && c >= 0 && c < cols) {  
        setPixelColor(r, c, currentColor);  
        applySymmetry(r, c, currentColor);  
        repaint();  
    }  
}
```

```
private void setPixelColor(int r, int c, Color color) {  
    grid[r][c] = color;  
}
```

```

private void applySymmetry(int r, int c, Color color) {
    if (symmetryMode.equals("Horizontal") || symmetryMode.equals("XY")) {
        grid[rows - 1 - r][c] = color;
    }
    if (symmetryMode.equals("Vertical") || symmetryMode.equals("XY")) {
        grid[r][cols - 1 - c] = color;
    }
    if (symmetryMode.equals("XY")) {
        grid[rows - 1 - r][cols - 1 - c] = color;
    }
}

```

```

public void resizeGrid(int newRows, int newCols) {
    Color[][] newGrid = new Color[newRows][newCols];
    for (int r = 0; r < newRows; r++) {
        for (int c = 0; c < newCols; c++) {
            if (r < this.rows && c < this.cols) {
                newGrid[r][c] = this.grid[r][c];
            } else {
                newGrid[r][c] = Color.WHITE;
            }
        }
    }
    this.rows = newRows;
    this.cols = newCols;
    this.grid = newGrid;
    setPreferredSize(new Dimension(cols * cellSize, rows * cellSize));
    revalidate();
    repaint();
}

```

```

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            g.setColor(grid[r][c]);
            g.fillRect(c * cellSize, r * cellSize, cellSize, cellSize);
            g.setColor(Color.LIGHT_GRAY);
            g.drawRect(c * cellSize, r * cellSize, cellSize, cellSize);
        }
    }
}

```

```

    public BufferedImage toImage(boolean includeGrid) {
        BufferedImage img = new BufferedImage(cols * cellSize, rows * cellSize,
        BufferedImage.TYPE_INT_ARGB);
        Graphics2D g2d = img.createGraphics();
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                g2d.setColor(grid[r][c]);
                g2d.fillRect(c * cellSize, r * cellSize, cellSize, cellSize);
                if (includeGrid) {
                    g2d.setColor(Color.LIGHT_GRAY);
                    g2d.drawRect(c * cellSize, r * cellSize, cellSize, cellSize);
                }
            }
        }
        g2d.dispose();
        return img;
    }

    public void setCurrentColor(Color color) { this.currentColor = color; }
    public void setSymmetryMode(String mode) { this.symmetryMode = mode; }
    public Color[][] getGridData() { return grid; }
    public void setGridData(Color[][] data) { this.grid = data; repaint(); }
}

```

Java

```
package serialization;
```

```
import java.awt.Color;
import java.io.*;
import java.util.regex.Pattern;
```

```
public class PatternSerializer {
    private static final int SUPPORTED_SCHEMA_VERSION = 1;
    private static final Pattern HEX_VALIDATOR = Pattern.compile("#[0-9A-Fa-f]{6}");

```

```

public static void save(EmbroideryPattern pattern, File file) throws IOException {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(file))) {
        writer.write("{\n");
        writer.write("  \"version\": " + SUPPORTED_SCHEMA_VERSION + ",\n");
        writer.write("  \"rows\": " + pattern.rows() + ",\n");
        writer.write("  \"cols\": " + pattern.cols() + ",\n");
        writer.write("  \"matrix\": [\n");
        for (int r = 0; r < pattern.rows(); r++) {
            writer.write("    [");
            for (int c = 0; c < pattern.cols(); c++) {
                writer.write("\"" + pattern.matrix()[r][c] + "\"");
                if (c < pattern.cols() - 1) writer.write(", ");
            }
            writer.write("]");
            if (r < pattern.rows() - 1) writer.write(",\n");
        }
        writer.write("\n  ]\n");
    }
}

```

```

public static EmbroideryPattern load(File file) throws IOException {
    StringBuilder jsonBuilder = new StringBuilder();
    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
        while ((line = reader.readLine()) != null) {
            jsonBuilder.append(line);
        }
    }
    return new JsonParser(jsonBuilder.toString()).parse();
}

```

```

private static class JsonParser {
    private final String src;
    private int idx = 0;

```

```

    public JsonParser(String src) { this.src = src; }

```

```

    public EmbroideryPattern parse() {
        skipWhitespace();
        if (src.charAt(idx) != '{') throw new IllegalArgumentException("Invalid JSON schema structure");
        idx++;

```

```

        int version = 0, rows = 0, cols = 0;
        String[][] matrix = null;

        while (idx < src.length()) {
            skipWhitespace();
            if (src.charAt(idx) == '}') break;
            if (src.charAt(idx) == ',') { idx++; continue; }

            String key = readString();
            skipWhitespace();
            if (src.charAt(idx) != ':') throw new IllegalArgumentException("Expected character
            ':'");
            idx++;

            switch (key) {
                case "version" -> version = Integer.parseInt(readNumber());
                case "rows" -> rows = Integer.parseInt(readNumber());
                case "cols" -> cols = Integer.parseInt(readNumber());
                case "matrix" -> matrix = parseMatrix(rows, cols);
                default -> skipValue();
            }
        }

        if (version != SUPPORTED_SCHEMA_VERSION) throw new
        IllegalStateException("Unsupported file version");
        return new EmbroideryPattern(version, rows, cols, matrix);
    }

    private String[][] parseMatrix(int rows, int cols) {
        skipWhitespace();
        if (src.charAt(idx) != '[') throw new IllegalArgumentException("Expected structure '[' for
        matrix");
        idx++;
        String[][] mat = new String[rows][cols];
        for (int r = 0; r < rows; r++) {
            skipWhitespace();
            if (src.charAt(idx) != '[') throw new IllegalArgumentException("Expected row
            encapsulation '['");
            idx++;
            for (int c = 0; c < cols; c++) {
                String color = readString();
                if (!HEX_VALIDATOR.matcher(color).matches()) color = "#FFFFFF";
                mat[r][c] = color;
            }
        }
    }

```

```

        skipWhitespace();
        if (src.charAt(idx) == ',') idx++;
    }
    if (src.charAt(idx) != ']') idx++;
    skipWhitespace();
    if (src.charAt(idx) == ',') idx++;
}
if (src.charAt(idx) != ']') idx++;
return mat;
}

```

```

private String readString() {
    skipWhitespace();
    if (src.charAt(idx) != '"') throw new IllegalArgumentException("Expected quote
character");
    idx++;
    StringBuilder sb = new StringBuilder();
    while (src.charAt(idx) != '"') {
        sb.append(src.charAt(idx));
        idx++;
    }
    idx++;
    return sb.toString();
}

```

```

private String readNumber() {
    skipWhitespace();
    StringBuilder sb = new StringBuilder();
    while (Character.isDigit(src.charAt(idx))) {
        sb.append(src.charAt(idx));
        idx++;
    }
    return sb.toString();
}

```

```

private void skipWhitespace() {
    while (idx < src.length() && Character.isWhitespace(src.charAt(idx))) idx++;
}

```

```

private void skipValue() {
    skipWhitespace();
    if (src.charAt(idx) == '"') { readString(); }
    else if (Character.isDigit(src.charAt(idx))) { readNumber(); }
}

```

}
}
}